



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №6
Технології розробки програмного забезпечення
«ШАБЛОНИ «Abstract Factory»,
«Factory Method», «Memento»,
«Observer», «Decorator»»

Виконав:
студент групи ІА-24
Іскандаров Е. Е.

Перевірив:
Мягкий М. Ю.

Тема: «ШАБЛЮНИ «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator»»

Зміст

Короткі теоретичні відомості.....	2
Реалізація шаблону проектування для майбутньої системи.....	3
Зображення структури шаблону	7

Короткі теоретичні відомості

Шаблон " Abstract Factory" використовується для створення сімейств взаємопов'язаних або залежних об'єктів без уточнення їх конкретних класів. Він визначає спільний інтерфейс для фабрик, які створюють об'єкти, а конкретні реалізації цього інтерфейсу відповідають за створення об'єктів певного сімейства.

Шаблон " Factory Method" визначає інтерфейс для створення об'єктів певного базового типу, дозволяючи замінювати їх підтипами без зміни основної логіки програми. Це забезпечує гнучкість і розширюваність системи, зберігаючи узгодженість поведінки об'єктів.

Шаблон " Memento" дозволяє зберігати і відновлювати стан об'єктів, не порушуючи інкапсуляцію, завдяки використанню спеціального об'єкта-зберігача стану. Логіка управління станами переноситься на зовнішній об'єкт (Caretaker), що забезпечує гнучке управління історією змін.

Шаблон " Observer" визначає залежність "один-до-багатьох", де зміна стану одного об'єкта автоматично сповіщає всі пов'язані об'єкти. Це забезпечує синхронізацію станів між об'єктами, що особливо корисно для реалізації складних систем.

Шаблон " Decorator" дозволяє динамічно додавати нову функціональність об'єкту, обгортаючи його в спеціальний об'єкт-декоратор. Це зберігає початкову поведінку об'єкту та забезпечує гнучкість у зміні функцій без зміни його структури.

Хід роботи:

Тема:

..6 Web-browser (proxy, chain of responsibility, factory method, template method, visitor, p2p)

Веб-браузер повинен мати можливість зробити наступне: мати адресний рядок для введення адреси сайту, переміщатися і відображати структуру html документа, переглядати підключений javascript та css файли, перегляд всіх підключених ресурсів (зображень), коректна обробка відповідей з сервера (коди відповідей HTTP) - переходи при перенаправленнях, відображення сторінок 404 і 502/503.

Веб-браузер, як складна система, повинен вміти завантажувати різні типи ресурсів (HTML, CSS, JavaScript, зображення тощо), кожен з яких має свої унікальні особливості та потребує специфічної логіки обробки. Враховуючи, що типи ресурсів можуть змінюватися або розширюватися в майбутньому, ідеальним рішенням для організації створення об'єктів є використання шаблону **Factory Method**.

Цей шаблон дозволяє централізувати логіку створення об'єктів, де кожен тип ресурсу відповідає своїй фабриці. У результаті код стає більш гнучким і відкритим для розширення: якщо потрібно додати підтримку нового типу ресурсу, достатньо створити нову фабрику і відповідний клас.

Наприклад, для HTML-ресурсів фабрика створює об'єкт, який відповідає за завантаження та обробку HTML-документів. Для CSS — об'єкт, що завантажує стилі. Аналогічно для JavaScript та зображень. Це дозволяє ізолювати логіку завантаження кожного типу ресурсу і підтримувати її незалежність, що полегшує тестування та розширення.

Таким чином, **Factory Method** забезпечує:

- **Гнучкість** у додаванні нових типів ресурсів.
- **Розширюваність** системи без зміни існуючого коду.
- **Інкапсуляцію логіки створення об'єктів**, що спрощує підтримку коду.

```
3  🚀 abstract class WebResource { 1 usage 4 inheritors
4  🦋   public abstract void load(); no usages 4 implementations
5  }
```

Рис. 1. Код абстрактного класу WebResource

Абстрактний клас, що представляє загальну структуру ресурсу веб-браузера. Містить метод load, який реалізується в підкласах для завантаження ресурсу.

```

3      class HtmlResource extends WebResource { 1 usage
4          @Override
5      public void load() {
6          System.out.println("Loading HTML resource...");
7      }
8  }
9

```

Рис. 2 – Код класу HtmlResource

```

3      class CssResource extends WebResource { 1 usage
4          @Override
5      public void load() {
6          System.out.println("Loading CSS resource...");
7      }
8  }
9

```

Рис. 3 – Код класу CssResource

```

3      class JavaScriptResource extends WebResource { no usages
4          @Override
5      public void load() {
6          System.out.println("Loading JavaScript resource...");
7      }
8  }
9

```

Рис. 4 – Код класу JavaScriptResource

```

3      class ImageResource extends WebResource { 1 usage
4          @Override
5      public void load() {
6          System.out.println("Loading Image resource...");
7      }
8  }
9

```

Рис. 5 – Код класу ImageResource

На рисунках 2-5 зображені Конкретні класи ресурсів (HTML, CSS, JavaScript, зображення). Реалізують метод load, визначаючи специфічну поведінку для кожного типу ресурсу.

```

3  abstract class WebResourceFactory { no usages 4 inheritors
4      public abstract WebResource createResource(); no usages 4 implementations
5  }

```

Рис. 6 – Код абстрактного класу WebResourceFactory

Абстрактний клас-фабрика. Містить метод createResource, який реалізується підкласами для створення конкретних ресурсів.

```

3      class HtmlResourceFactory extends WebResourceFactory { 1 usage
4          @Override 4 usages
5      public WebResource createResource() {
6          return new HtmlResource();
7      }
8  }

```

Рис. 7 – Код класу HtmlResourceFactory

```

3      class CssResourceFactory extends WebResourceFactory { no usages
4          @Override no usages
5      public WebResource createResource() {
6          return new CssResource();
7      }
8  }

```

Рис. 8 – Код класу CssResourceFactory

```

3      class JavaScriptResourceFactory extends WebResourceFactory { 1 usage
4          @Override 4 usages
5      public WebResource createResource() {
6          return new JavaScriptResource();
7      }
8  }

```

Рис. 9 – Код класу JavaScriptResourceFactory

```

3      class ImageResourceFactory extends WebResourceFactory { no usages
4          @Override no usages
5      public WebResource createResource() {
6          return new ImageResource();
7      }
8  }

```

Рис. 10 – Код класу ImageResourceFactory

На рисунках 7-10 зображені конкретні фабрики для створення ресурсів відповідного типу. Визначають, який саме об'єкт буде створено.

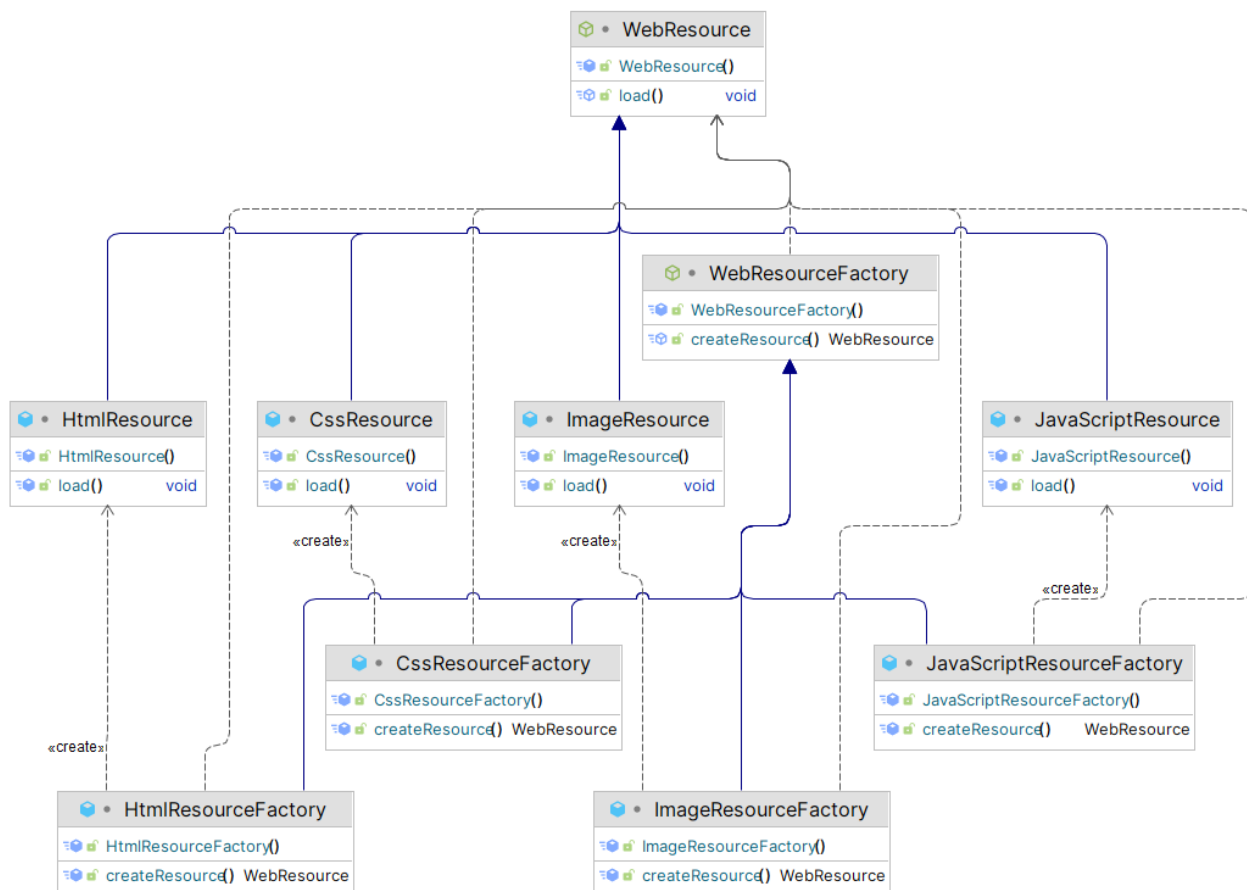


Рис. 11 - Структура шаблону Factory Method

Шаблон **Factory Method** є прикладом ефективної розробки, оскільки забезпечує інкапсуляцію логіки створення об'єктів, роблячи систему гнучкою, розширюваною та легко підтримуваною.

Використання цього шаблону дозволяє:

1. **Спрощувати розширення:** Додавання нового типу об'єкта (наприклад, нового типу ресурсу в веб-браузері) не потребує зміни існуючого коду, а лише створення нового класу-фабрики. Це відповідає принципу відкритості/закритості (ОСР).
2. **Зменшувати залежності:** Клієнтський код не залежить від конкретних класів об'єктів, що спрощує заміну або модифікацію об'єктів.
3. **Покращувати тестування:** Оскільки логіка створення ізольована у фабриках, її можна легко тестувати окремо від решти системи.
4. **Структурувати код:** Створення кожного типу об'єкта організоване у власній фабриці, що забезпечує чіткий поділ відповідальностей.

Завдяки цьому шаблону розробка стає більш модульною та масштабованою, що особливо важливо для систем із динамічними вимогами, такими як веб-браузери. **Factory Method** демонструє, як грамотно організувати створення об'єктів,

зберігаючи при цьому чистоту коду та можливість його розширення в майбутньому.

Реалізація: https://github.com/Elmir2/trpz_labs

Висновок: Під час виконання даної лабораторної роботи було розглянуто структуру, призначення, переваги та недоліки патернів проєктування «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator». Розглянуті шаблони програмування мають свої власні переваги та недоліки, а також використовуються для досягнення різних цілей. Для реалізації частини майбутньої системи було обрано шаблон Factory Method, який найкраще підійде у даному випадку. Далі було реалізовано обраний патерн, розглянуто його структуру та досліджено його переваги для використання для майбутньої системи.